Reflection Template

Use Case	UC-4
Use Case Name	View Project
Requirement IDs	FR4, FR20, FR22, FR23
Matching Feature Comparison Sets	CS-10
Implementation Order	Kotlin first -> Java second

Feature Relevance

Feature/s:

Extension functions

Did the feature/s occur naturally in this/these context/s?

Yes, findByIdOrThrow was reused from Extensions.kt without any changes it just worked. Java the same orElseThrow pattern had to be written again inline.

Qualitative Ratings (1-5)

- 1 = very poor/very difficult
- 2 = rather poor/difficult
- 3 = neutral/moderate
- 4 = good/easy
- 5 = very good/very easy

Criteria	Java	Kotlin
Readability	4	4
Compactness of logic	3	4
Perceived implementation effort	5	5

Justifications

Readability:

Pretty equal cause it’s just fetching the ID (get /projects)

Compactness of logic:

Kotlin is slightly more compact because findByIdOrThrow hides the error handling logic. Java's orElseThrow inline is still readable but slightly more verbose each time it appears.

Implementation effort:

Both really easy, same pattern as UC-2. Implementing Kotlin first didn't really make a difference.

Observed structural differences

findByIdOrThrow is reused in Kotlin without modification beacuse of extension function. Java writes the same orElseThrow pattern for the third time now.

No new structural differences compared to UC-2, this use case confirms the existing patterns rather than introducing new ones.

Final comparative summary

UC-4 is a short use case that mostly confirms what was already observed in UC-2. The extension function continues to pay off in Kotlin while Java repeats the inline pattern. No new language differences emerged here.